Evaluación de producto: Bubble Sort vs. Merge Sort

Comparación experimental de dos algoritmos de ordenamiento

1. Objetivo del proyecto

El objetivo de este proyecto es comparar el comportamiento de Bubble Sort y Merge Sort mediante pruebas realizadas en C++. Para ello, se utilizan diferentes cantidades de elementos y distintos tipos de entrada, con el fin de observar cómo cambia el rendimiento de cada algoritmo.

Durante las pruebas se registran el tiempo de ejecución, las comparaciones entre elementos, los intercambios o movimientos y la profundidad de recursión cuando corresponde. Luego, estos datos se utilizan para elaborar tablas y gráficos que permitan comparar los resultados obtenidos con la complejidad teórica de ambos algoritmos.

El código y los archivos generados se organizan en un repositorio de GitHub para facilitar su consulta y reproducción.

2. Algoritmos utilizados

Para el experimento se eligieron dos algoritmos que resuelven el mismo problema mediante procedimientos diferentes.

Bubble Sort. Compara elementos que se encuentran uno al lado del otro y los intercambia cuando están en el orden incorrecto. Es sencillo de implementar y no necesita un arreglo auxiliar, aunque su cantidad de comparaciones aumenta considerablemente cuando crece el tamaño de la entrada.

Merge Sort. Divide el arreglo en partes más pequeñas, ordena cada parte y finalmente combina los resultados. Su rendimiento temporal es más favorable para entradas grandes, pero necesita memoria auxiliar y utiliza llamadas recursivas.

La Tabla 1 resume las principales características de las implementaciones consideradas.

Tabla 1
Características teóricas de los algoritmos

Característica	Bubble Sort	Merge Sort
Estrategia	Comparaciones e intercambios adyacentes	División y combinación de arreglos
Mejor caso	$O(n^2)$	$O(n \log n)$
Caso promedio	$O(n^2)$	$O(n \log n)$
Peor caso	$O(n^2)$	$O(n \log n)$
Memoria auxiliar	$O(1)$	$O(n)$
Estabilidad	Estable si solo intercambia cuando $A[j] > A[j+1]$	Estable si se elige primero el elemento izquierdo cuando las claves son iguales
Recursión	No utiliza	Sí utiliza

En las pruebas se utiliza una implementación de Bubble Sort sin parada anticipada. Por su parte, Merge

Sort emplea una versión recursiva que utiliza un arreglo auxiliar para combinar los elementos ordenados.

3. Diseño experimental

Las pruebas se realizan con diferentes tamaños de entrada y cinco patrones de datos. De esta manera, no se evalúa únicamente el caso de una lista desordenada al azar, sino también el comportamiento de los algoritmos cuando los datos ya presentan cierto orden.

3.1. Patrones de entrada

Se consideran los siguientes patrones:

Tabla 2
Tipos de entrada utilizados en el experimento

Patrón	Descripción
Aleatorio	Números enteros generados uniformemente entre 1 y 1 000 000.
Ascendente	Elementos colocados de menor a mayor: $1, 2, \ldots, n$.
Descendente	Elementos colocados de mayor a menor: $n, n-1, \ldots, 1$.
Casi ordenado	Arreglo inicialmente ascendente al que se le realizan $n/40$ intercambios aleatorios.
Con duplicados	Arreglo formado con valores repetidos entre 1 y 15.

3.2. Tamaños y repeticiones

Se consideran tres tamaños de entrada:

$$n \in \{1000,\ 10000,\ 100000\}.$$

Cada combinación de algoritmo, patrón y tamaño se repite 30 veces, excepto las pruebas de Bubble Sort con 100 000 elementos, que se omiten debido al elevado costo de ejecución.

En ese tamaño, Bubble Sort puede realizar cerca de 5 000 millones de comparaciones por ejecución. Por este motivo, se limita su evaluación a 1 000 y 10 000 elementos, mientras que Merge Sort se prueba con los tres tamaños.

3.3. Generación de los datos

Para facilitar la repetición del experimento, se utiliza un generador congruencial lineal propio, en lugar de depender de `rand()`. La semilla de cada repetición se obtiene a partir de la semilla base 12345.

La semilla utilizada en la repetición r es:

$$\text{semilla} = 12345 + r.$$

Por tanto, las 30 repeticiones utilizan semillas desde 12346 hasta 12375.

En cada repetición, ambos algoritmos reciben los mismos datos de entrada para que la comparación se realice bajo condiciones equivalentes.

4. Métricas registradas

El programa guarda los resultados de cada ejecución en un archivo CSV. Cada fila corresponde a una prueba individual e incluye los datos necesarios para identificarla y analizarla.

Tabla 3
Información registrada en el archivo CSV

Columna	Descripción
`algoritmo`	Identifica Bubble Sort o Merge Sort.
`n`	Cantidad de elementos de la entrada.
`patron`	Tipo de entrada utilizado.
`repeticion`	Número de repetición, del 1 al 30.
`tiempo_ms`	Tiempo de ordenamiento en milisegundos, medido con `std::chrono::steady\_clock`.
`comparaciones`	Número de comparaciones realizadas entre elementos.
`intercambios`	En Bubble Sort registra intercambios de pares. En Merge Sort registra movimientos al arreglo auxiliar.
`profundidad\_recursion`	Profundidad máxima registrada en Merge Sort. En Bubble Sort se registra 0.
`semilla`	Semilla empleada para generar los datos.
`commit\_hash`	Identificador de la versión del código utilizada.

El tiempo se mide únicamente durante el ordenamiento. La generación de los datos, la verificación del resultado y la escritura del archivo CSV quedan fuera del intervalo cronometrado.

Después de cada ejecución se comprueba que el arreglo esté ordenado correctamente. Los contadores de operaciones se mantienen durante las pruebas, por lo que su actualización también forma parte del trabajo realizado dentro del algoritmo medido.

5. Entorno de ejecución

Las pruebas se realizaron en el siguiente equipo:

Componente	Especificación
Procesador	Intel Core i5-7400, 3,00 GHz.
Núcleos e hilos	4 núcleos y 4 hilos.
Memoria RAM	8 GB DDR4 a 2133 MHz.
Sistema operativo	Windows 11.
Compilador	TDM-GCC 4.9.2 de 64 bits, desde Dev-C++.
Estándar de C++	C++11.
Optimización	Sin la opción `-O2`.

Se procuró mantener las mismas condiciones de ejecución para ambos algoritmos, cerrando los demás programas durante la corrida final. Los tiempos obtenidos corresponden a este equipo y pueden variar si el experimento se ejecuta en otra computadora.

6. Estructura del repositorio

El proyecto se organiza en carpetas para distinguir el código, la documentación y los resultados del experimento.

```
.
|-- README.md
|-- .gitignore
|-- analisis.py
|-- docs/
|   |-- manuscrito.pdf
|   \-- diapositiva.pdf
|-- include/
|-- src/
|   \-- main.cpp
|-- data/
\-- results/
    |-- resultados.csv
    |-- tabla_tiempos_bubble.csv
    |-- tabla_tiempos_merge.csv
    |-- tabla_contadores_bubble.csv
    |-- tabla_contadores_merge.csv
    |-- fig1_n_vs_tiempo.png
    |-- fig2_boxplot.png
    \-- fig3_contadores.png
```

Los archivos principales son:

- **`src/main.cpp`:** contiene los algoritmos, la generación de entradas y las pruebas de rendimiento.
- **`results/resultados.csv`:** almacena las mediciones obtenidas en las ejecuciones.
- **`analisis.py`:** procesa los datos y genera las tablas y figuras del análisis.
- **`docs/`:** reúne el artículo científico y la diapositiva de presentación.
- **`include/`:** se reserva para archivos de cabecera propios, si se decide separar las interfaces del código principal.

- **`data/`:** se reserva para archivos de entrada si posteriormente se decide almacenarlos por separado.

Las carpetas `include/` y `data/` no son necesarias para la versión que mantiene el programa en un único archivo y genera los datos en memoria. Se incluyen en la estructura prevista del repositorio para una posible ampliación.

7. Compilación y ejecución

7.1. Desde Dev-C++

Se abre el archivo `src/main.cpp`, se comprueba que el compilador utilice C++11 y se reconstruye el proyecto antes de ejecutarlo.

7.2. Desde la terminal

Desde la raíz del repositorio, se puede compilar el programa con el siguiente comando:

```
g++ -std=c++11 src/main.cpp -o benchmark
```

En Linux, la ejecución se realiza con:

```
./benchmark
```

En Windows, si el ejecutable se llama `benchmark.exe`, se puede ejecutar con:

```
benchmark.exe
```

El programa genera el archivo `resultados.csv` en la carpeta de trabajo desde la que se ejecuta. Para mantener la organización del repositorio, se debe colocar el archivo generado en `results/resultados.csv` antes de realizar el análisis, si el programa no lo guarda directamente allí.

Para comprobar el funcionamiento sin ejecutar todas las pruebas, se pueden reducir temporalmente las repeticiones y los tamaños en el código. Antes de obtener los resultados definitivos, se deben restaurar los valores establecidos para el experimento completo.

8. Análisis de los resultados

El análisis se realiza con Python 3 y las bibliotecas `pandas` y `matplotlib`. Para instalarlas, se ejecuta:

```
pip install pandas matplotlib
```

Después, desde la raíz del repositorio, se ejecuta:

```
python analisis.py
```

El script lee `results/resultados.csv` y genera tablas resumidas y figuras que permiten examinar las diferencias entre ambos algoritmos. También puede recibir la ruta de otro archivo CSV como argumento:

```
python analisis.py ruta/al/archivo.csv
```

Las tablas permiten resumir los tiempos y los contadores, mientras que las figuras facilitan la comparación entre tamaños de entrada y patrones. Para interpretar los tiempos, se consideran medidas descriptivas como la mediana, la media, la desviación estándar, el mínimo y el máximo.

9. Resultados principales

Para una entrada de 10 000 elementos, se obtuvieron las siguientes medianas de tiempo, expresadas en milisegundos.

Tabla 4
Mediana del tiempo de ejecución para $n = 10\,000$

Patrón	Bubble Sort (ms)	Merge Sort (ms)
Aleatorio	285,4	1,75
Ascendente	125,5	0,85
Descendente	222,0	0,88
Casi ordenado	147,7	0,93
Con duplicados	286,9	1,24

En los datos aleatorios de este tamaño, la mediana registrada para Bubble Sort fue aproximadamente 163 veces mayor que la de Merge Sort. Esta diferencia muestra el efecto que puede tener la complejidad temporal cuando aumenta la cantidad de elementos.

En la implementación utilizada, Bubble Sort realiza $n(n-1)/2$ comparaciones independientemente del patrón de entrada, ya que no cuenta con una condición de parada temprana. Sin embargo, la cantidad de intercambios sí cambia: es cero para una entrada ascendente y alcanza $n(n-1)/2$ para una entrada estrictamente descendente.

Por su parte, Merge Sort mantiene un crecimiento temporal teórico de $O(n \log n)$. En las pruebas con 100 000 elementos, sus medianas se situaron entre 10 y 20 ms, según el patrón de entrada.

Estos resultados corresponden a las mediciones registradas en el equipo descrito anteriormente. No deben interpretarse como tiempos universales, ya que pueden cambiar según el hardware, el compilador y las condiciones de ejecución.

10. Reproducibilidad y versión del código

Para facilitar la repetición del experimento, se conservan la configuración de las pruebas, las semillas y el archivo CSV con los datos obtenidos.

- **Semilla base:** 12345.
- **Repeticiones:** 30 por combinación evaluada.
- **Datos registrados:** una fila por ejecución.

- **Versión del código:** debe identificarse mediante el hash real del commit utilizado para generar los resultados.

El archivo de resultados se atribuye al commit `2c4e2dc`, según el registro del proyecto. Antes de publicarlo, se debe comprobar que ese identificador corresponda realmente a la versión utilizada para obtener los datos.

Con el mismo código y las mismas semillas se pueden reproducir las entradas generadas. Los tiempos de ejecución, en cambio, pueden variar entre equipos y condiciones de prueba.